COURSE: OPERATING SYSTEM LAB

Project Review 2

MEMORY AND SWAP OPTIMIZER AGENT

Submitted by

S.No.	Name	Registration Number
1	Kumar Hrishikesh Ranjan	24BDS0221
2	Ashish Kumar	24BKT059

COURSE CODE: BCSE303P

FACULTY: NAVEENKUMAR J

SLOT: L53+L54

Review 2

Chapter 3: System Architecture & Design

This chapter describes how the proposed LLM-powered Operating System Agent is structured. The system lets a user control a Linux machine using plain English. A Large Language Model (LLM) interprets the request and chooses a suitable tool, while a separate, security-hardened layer actually runs the command. The LLM never touches the shell directly. It can only request actions from a fixed set of whitelisted tools, which keeps the design predictable and safe.

3.1 High-Level Architecture

The core of the system is the **User-to-Shell loop**: User Input -> LLM Processing -> Tool Call -> System Execution -> Feedback. Every request passes through the same five stages, and the result of each execution is fed back to the LLM so it can explain the outcome or decide on a follow-up action.

1. **User Input:** the user types a natural-language request at the command line, for example *"My computer is lagging. Free up some RAM cache and check for memory leaks."*
2. **LLM Processing:** the LLM receives the request together with the list of available tools and their schemas. It works out the intent and decides which tool, with which arguments, fits best.
3. **Tool Call:** instead of free text, the LLM emits a structured JSON tool call. The MCP client forwards this to the MCP server over the Model Context Protocol.
4. **System Execution:** the MCP server validates the call, applies the security checks of Section 3.3, and runs the approved command (for example `systemctl`, `lsmod`, `sha256sum`).
5. **Feedback:** the captured output (stdout, stderr, exit code) is formatted and returned to the LLM, which turns it into a clear explanation for the user. If more steps are needed, the loop repeats.

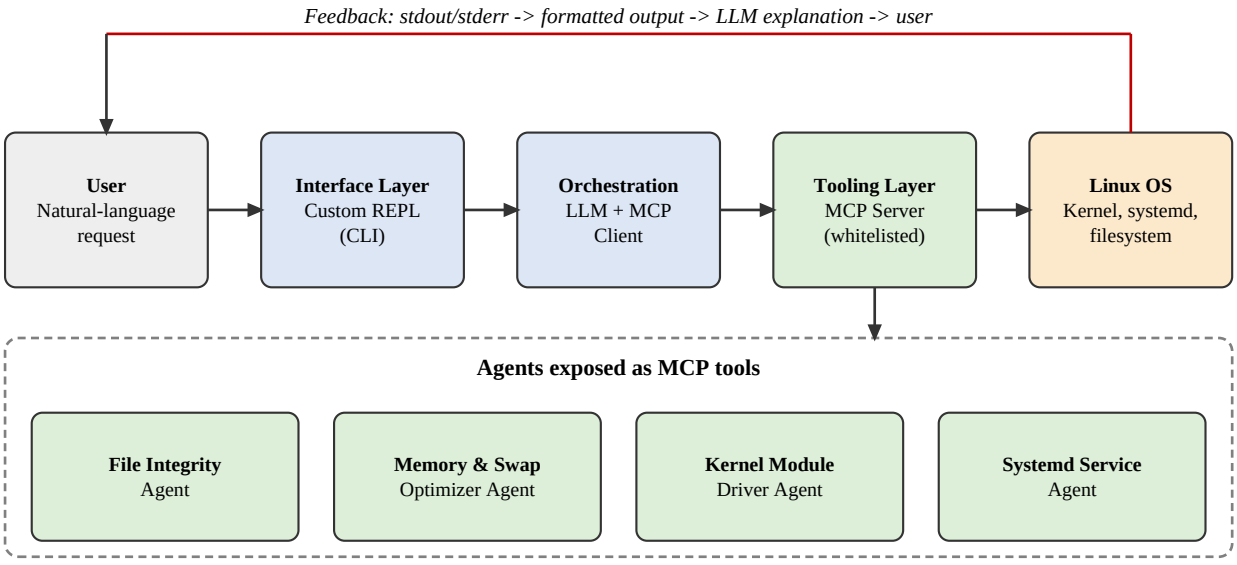


Fig. 3.1: High-level block diagram of the LLM-powered OS agent

3.2 Component Design

The system follows a three-layer design. Each layer has a single responsibility and talks only to its neighbours, which makes the layers independent, testable, and replaceable (for instance, the LLM provider can be swapped without changing the tools).

- **The Interface Layer:** a custom REPL (Read-Eval-Print Loop) environment. It reads user input, keeps session history, prints the LLM's answers, and displays confirmation prompts for risky actions. It contains no system logic.
- **The Orchestration Layer:** LangChain/LlamaIndex or a raw MCP client handling intent routing. It builds the prompt, sends it to the LLM, parses the returned tool call, forwards it to the MCP server, and feeds the result back. It also maintains conversation memory.
- **The Tooling Layer (MCP Server):** exposes file management, system monitoring, and network configuration utilities as typed tools. Every tool has a name, a description, and a JSON schema, so the LLM can only call what is explicitly registered.

The Tooling Layer is organised into agents, each covering one area of the operating system:

Agent	Example prompt	Underlying tools	Risk
File Integrity Agent	"Create hashes of my critical files and check them later."	sha256sum, hidden JSON hash store, hash diff	Low
Memory & Swap Optimizer Agent	"My computer is lagging. Free up some RAM cache and check for memory leaks."	write 3 to /proc/sys/vm/drop_caches, smem, top, free	Mediu m
Kernel Module Driver Agent	"My webcam isn't working. Try reloading the video drivers."	lsmod, rmmod, modprobe (e.g., uvcvideo)	High
Systemd Service Agent	"Restart the database service and make sure it starts on boot."	systemctl restart / enable / status	High

3.3 Security & Guardrail Framework

Giving an AI model access to a shell is risky: the LLM may misunderstand a request or hallucinate a command. The framework therefore uses defence in depth, with several independent checks, so no single failure can cause damage. Fig. 3.2 shows the path every tool call must follow.

- **The "Human-in-the-Loop" validation mechanism:** read-only tools (such as listing a directory or showing memory usage) run automatically. Any state-changing action (restarting a service, unloading a kernel module, dropping caches) is shown to the user in full and runs only after an explicit y/n approval.
- **Command sanitization and regex-based blacklisting of destructive commands:** patterns such as `rm -rf /`, `mkfs`, `dd if=`, fork bombs, and redirection to raw disks (`> /dev/sda`) are rejected. Arguments are checked against the tool schema, and commands are passed as argument lists, never as concatenated shell strings, to prevent injection.
- **Least privilege:** the agent runs as a normal user. `sudo` is allowed only for a short whitelist of commands.
- **Sandboxed execution:** every command runs with a timeout and output size limit, so a hung or noisy process cannot freeze the agent.

- **Audit logging:** each request, tool call, approval decision, and result is written to a log file for traceability.

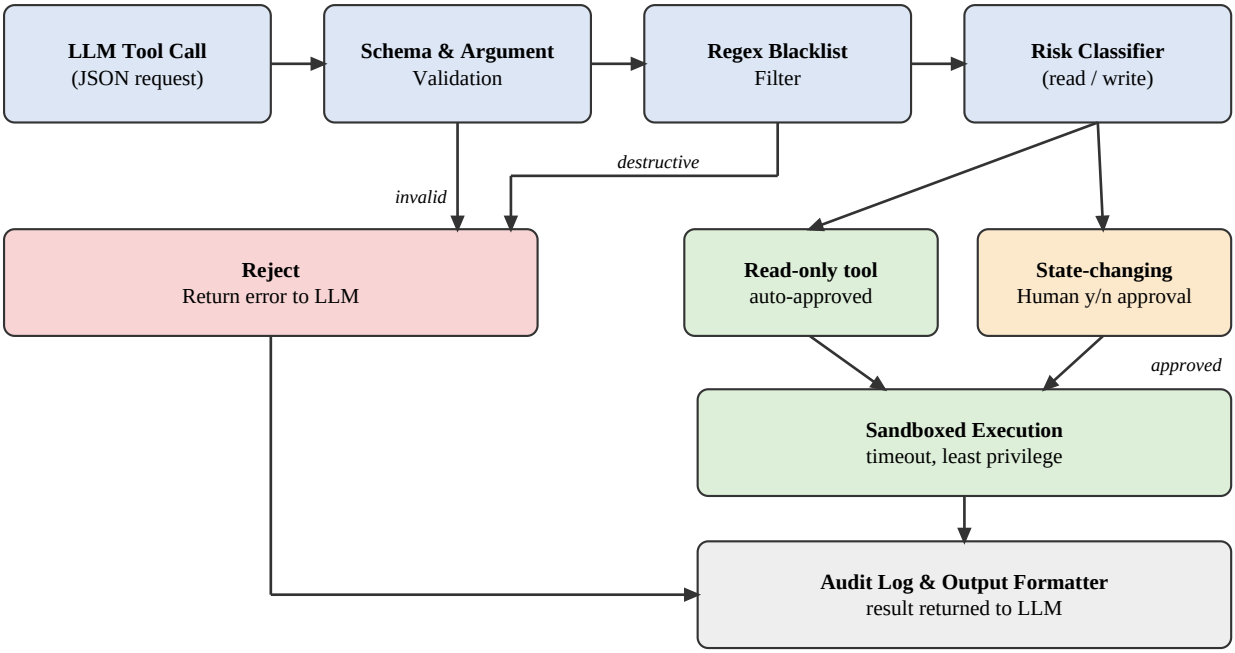


Fig. 3.2: Security and guardrail pipeline for every tool call

Chapter 4: Implementation Methodology

This chapter explains the tools chosen for building the system and the order in which it is developed. The build is incremental: each step produces something that can be run and tested before the next one begins.

4.1 Development Environment & Tech Stack

Component	Choice	Reason
Programming Language	Python 3 (Node.js as an alternative)	Rich system libraries (subprocess, psutil, hashlib) and an official MCP SDK.
LLM Provider	OpenAI API / Anthropic Claude / Local Ollama (e.g., Llama 3)	Cloud models give the best tool-calling accuracy. A local Ollama model works offline and keeps system data private.
SDK	MCP Python SDK	Standard way to define and expose tools to any LLM client.
Operating System	Ubuntu Linux	Provides /proc, systemd, and kernel modules used by the agents.
Supporting Libraries	subprocess, psutil, hashlib, json, re	Command execution, system statistics, SHA-256 hashing, hash storage, and blacklist filtering.

4.2 Step-by-Step Implementation

Step 1: Setting up the base CLI prompt loop. Build the REPL that reads user input, keeps conversation history, handles exit commands, and prints responses. At this stage it only talks to the LLM, with no tools attached. This confirms that the API connection and the interface work.

Step 2: Building the MCP Server to expose safe system functions (e.g., list_directory, get_system_stats). The server registers each function with a typed schema and a clear description. Read-only tools are built first (list_directory, get_system_stats, verify_hashes). State-changing tools (service_control, module_control, drop_caches) are added later and are flagged as requiring approval.

Step 3: Prompt engineering for exact tool matching and JSON format enforcement. A system prompt tells the model to use only registered tool names, to return strict JSON, and to ask for clarification instead of guessing. Few-shot examples teach the mapping from requests to tool calls, for instance:

```
User: "My webcam isn't working. Try reloading the video drivers."
{"tool": "module_control",
 "arguments": {"module": "uvcvideo", "action": "reload"}}
```

Step 4: Implementing the command execution wrapper and output formatter. The wrapper runs each approved command through `subprocess` with an argument list, a timeout, and captured stdout/stderr. It applies the blacklist and approval checks from Section 3.3 and writes the audit log. The formatter trims long output and converts raw results into readable text before they go back to the LLM.